Resource-based authorization handlers are essentially the same as the authorization handler implementations you saw in section 24.4.2. The only difference is that the handler also has access to the resource being authorized.

To create a resource-based handler, you should derive from the `AuthorizationHandler<TRequirement, TResource>` base class, where `TRequirement` is the type of requirement to handle and `TResource` is the type of resource that you provide when calling `IAuthorizationService`. Compare this with the `AuthorizationHandler<T>` class you implemented previously, where you specified only the requirement.

The next listing shows the handler implementation for your recipe application. You can see that you've specified the requirement as `IsRecipeOwnerRequirement` and the resource as `Recipe`, and you have implemented the `HandleRequirementAsync` method.

Listing 24.15 `IsRecipeOwnerHandler` for resource-based authorization

```
public class IsRecipeOwnerHandler :
        AuthorizationHandler<IsRecipeOwnerRequirement, Recipe>     ❶
{
    private readonly UserManager<ApplicationUser> _userManager;    ❷
    public IsRecipeOwnerHandler(                                   ❷
        UserManager<ApplicationUser> userManager)                  ❷
    {                                                              ❷
        _userManager = userManager;                                ❷
    }                                                              ❷
    protected override async Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        IsRecipeOwnerRequirement requirement,
        Recipe resource)                                           ❸
    {
        var appUser = await _userManager.GetUserAsync(context.User);
        if(appUser == null)                                        ❹
```